Structure and Implement the Online Services Plugins

Guide to signing and implementing your Online Services plugin code.

Overview
What This Page Covers
Before You Begin
Configure
Structure
Add a Game Instance
Add a Game Instance Subsystem
Specify Your Project's Game Instance Class
Add a Player Controller
Add Code
Game Instance
Game Instance Subsystem
Implementation Details
Query and Set
Implementation Details
Online Services Interfaces
Implementation Details
Implementation Details
Implementation Details

```
CheckOnlineServicesInfoInternal(OnlineServicesType, OnlineServicesInfoInternal);

// Verify Service type
OnlineServicesInfoInternal ThisOnlineServicesType = OnlineServicesInfoInternal(OnlineServices
if (ThisOnlineServicesInfoInternal.IsValid())
{
  // Initialize Interface pics
  OnlineServicesInfoInternal AuthInterface = OnlineServicesInfoInternal(ThisOnlineservices
  CheckOnlineServicesInfoInternal(AuthInterface.IsValid());

  OnlineServicesInfoInternal TitleFileInterface = OnlineServicesInfoInternal(ThisOnlineSe
  CheckOnlineServicesInfoInternal(TitleFileInterface.IsValid());
}
else {
  UE_LOG(LogOnlineSampleOnlineSubsystem, Error, TEXT("Error: Failed to initialize servic
}
}

// <summary>
// Enumerate Files, GetNumcachedFiles, and ReadFile. This implementation uses a lambda fo
// to handle the delegate callback.
// </summary>
// <param name="Pinline">File to load</param>
// <param name="PlatformUserId">User to retrieve file for</param>
void OnlineSampleOnlineSubsystem::RetrieveTitleFileThing(Filename, PlatformUserId Pla
{
  using namespace UE::Online;

  FTitleFileRetrieveTitleFile: Params EnumParams;
  TRetrieveTitleFileRetrieveTitleFile: Result EnumResult;
  TOnlineResultOnlineInfo: Errorinfo;
  TOnlineServicesInfoInternalOnlineServicesInfoInternal;
  {
    OnlineServices = *GetOnlineServicesInterface(GetOnlineSubsystemName());
    LocalAccountId = OnlineUser->AccountId;
    EnumParams.LocalAccountId = LocalAccountId;
    if (OnlineServicesInfoInternal->TitleFileInterface.IsValid())
    {
      OnlineServicesInfoInternal->TitleFileInterface->EnumerateFiles(MoveTemp(EnumParams
    }
    else
    {
      UE_LOG(LogOnlineSampleOnlineSubsystem, Error, TEXT("Title File Interface pointer inv
    }
  }
  else
  {
    UE_LOG(LogOnlineSampleOnlineSubsystem, Error, TEXT("Could not find user with PlatfOrm
  }
}

// <summary>
// Register online user with local registry OnlineUserInfo
// </summary>
// <param name="PlatformUserIdPlatform user id of user to register"></param>
void OnlineSampleOnlineSubsystem::RegisterLocalOnlineUser(PlatformUserId PlatformUserId)
{
  using namespace UE::Online;

  FAuthRetrieveAccountByPlatformUserId: Params GetUserParams;
  GetUserParams.PlatformUserId = PlatformUserId;
  if (OnlineServicesInfoInternal->AuthInterface.IsValid())
  {
    OnlineServicesInfoInternal->AuthInterface->RetrieveAccountByPlatformUserId(MoveTemp(Ge
    {
      if (AuthGetResult.IsOk())
      {
        FAuthRetrieveLocalUserInfoOnlineUserInfo: Result LocalUserInstance = AuthGetResult.Get
        TMaybeBuilderAccountInfoUniquePtr GetAccountInfo = LocalUserInstance.AccountInfo;
        FAccountInfo GetAccountInfoPointer = *GetAccountInfo;
        if (!OnlineUserInfo.Contains(GetAccountInfoPointer.PlatformUserId))
        {
          OnlineUserInfo* Newuser = CreateAndRegisterUserInfo(GetAccountInfoPointer.Accour
          {
            UE_LOG(LogOnlineSampleOnlineSubsystem, Log, TEXT("Local User Registered: %s", *U
          }
        }
        else
        {
          UE_LOG(LogOnlineSampleOnlineSubsystem, Log, TEXT("Local User with platform user i
        }
      }
      else
      {
        FOnlineError ErrorResult = AuthGetResult.GetErrorValue();
        UE_LOG(LogOnlineSampleOnlineSubsystem, Error, TEXT("Get Local Online User Error: %s"
      }
    }
  }
  else
  {
    UE_LOG(LogOnlineSampleOnlineSubsystem, Error, TEXT("Auth Interface pointer invalid.")
  }
}

// <summary>
// Retrieve title file and read its contents.
// </summary>
// <param name="Pinline">File to load</param>
// <param name="PlatformUserId">User to read file for</param>
// <returns>FileString with contents of Filename</returns>
FString OnlineSampleOnlineSubsystem::ReadTitleFileThing(Filename, PlatformUserId Plat
{
  using namespace UE::Online;

  RetrieveTitleFileThing(Filename, PlatformUserId);
  FTitleFileContents FileContents = OnlineServicesInfoInternal->TitleFileContent;
  FString FileString = FString(FileContents.Num(), (TCHAR*)(FileContents.GetData()));
  UE_LOG(LogOnlineSampleOnlineSubsystem, Log, TEXT("Reading Title File: %s", *Filename);
  return FileString;
}

// <summary>
// Create a UOnlineUserInfo object from the given infoNumber.
// </summary>
// <param name="LocalUserIndex"></param>
// <param name="PlatformUserId">User</param>
// <param name="PlatformPlatform">Platform</param>
// <param name="Service">Online services user is registered with</param>
// <returns>Object pointer to the Newuser</returns>
TOnlineUserPtrOnlineUserInfo: OnlineSampleOnlineSubsystem::CreateOnlineUserInfo(int32 Local
{
  TOnlineUserPtrOnlineUserInfo: Newuser = NewObjectOnlineUserInfo<TOnlineUserInfo>();
  Newuser->LocalUserIndex = LocalUserIndex;
  Newuser->PlatformUserId = PlatformUserId;
  Newuser->AccountId = AccountId;
  Newuser->Services = Services;
  return Newuser;
}

// <summary>
// Create a UOnlineUserInfo object by calling CreateOnlineUserInfo then
//  register the user with the local registry OnlineUserInfo
// </summary>
// <param name="LocalUserIndex"></param>
// <param name="PlatformUserId">User</param>
// <param name="PlatformPlatform"></param>
// <param name="AccountId">User</param>
// <param name="Services">Online services user is registered with</param>
// <returns>Object pointer to the Newuser</returns>
TOnlineUserPtrOnlineUserInfo: OnlineSampleOnlineSubsystem::CreateAndRegisterUserInfo(int32
{
  TOnlineUserPtrOnlineUserInfo: Newuser = CreateOnlineUserInfo(LocalUserIndex, PlatformUser
  OnlineUserInfo.Add(PlatformUserId, Newuser);
  return Newuser;
}

// <summary>
// Get UOnlineUserInfo for provided platform user id.
// </summary>
// <param name="PlatformUserId">id of user to retrieve</param>
// <returns>Object pointer to OnlineUser</returns>
TOnlineUserPtrOnlineUserInfo: OnlineSampleOnlineSubsystem::GetOnlineUserInfo(PlatformUserI
{
  TOnlineUserPtrOnlineUserInfo: OnlineUser;
  if (OnlineUserInfo.Contains(PlatformUserId))
  {
    OnlineUser = *OnlineUserInfo.FindOrFindUserInfo();
  }
  else
  {
    UE_LOG(LogOnlineSampleOnlineSubsystem, Error, TEXT("Could not find user with PlatfOrm
    OnlineUser = nullptr;
  }
  return OnlineUser;
}

// <summary>
// Constructor for UOnlineUserInfo object
// </summary>
UOnlineUserInfo::UOnlineUserInfo()
{
}

// <summary>
// Return debug string for UOnlineUserInfo
// </summary>
// <returns>Debug representation of UOnlineUserInfo</returns>
const FString UOnlineUserInfo::DebugGetString()
{
  int32 UserIndex = this->LocalUserIndex;
  int32 PlatformId = this->PlatformUserId;
  TArrayFStringParamArgs ParamArgs;
  ParamArgs.Add(FStringFormatArgs(UserIndex));
  ParamArgs.Add(FStringFormatArgs(PlatformId));
  return FString::Format(TEXT("LocalUserNumber: {0}, PlatformUserId: {1}"), ParamArgs);
}
```

Copy full snippet

Player Controller

The player controller class is where you register your player with the online services and read the title file from the backend services.

OnlineSamplePlayerController.h

OnlineSamplePlayerController.h

```
#pragma once

#include "CoreMinimal.h"
#include "GameFramework/PlayerController.h"
#include "OnlineSamplePlayerController.generated.h"

/**
 *
 *
 */
UCLASS()
class ONLINESAMPLE_API AOnlineSamplePlayerController : public APlayerController
{
  GENERATED_BODY()

  public:

  AOnlineSamplePlayerController();

  protected:

  /** Called once play begins */
  virtual void BeginPlay();

  /** Called once play ends */
  virtual void EndPlay(EEndPlayReason::Type EndReason);
};
```

Copy full snippet

OnlineSamplePlayerController.cpp

OnlineSamplePlayerController.cpp

```
#include "OnlineSamplePlayerController.h"
#include "OnlineSampleGameInstance/OnlineSampleGameInstance.h"
#include "OnlineSampleGameInstance/OnlineSampleOnlineSubsystem.h"

AOnlineSamplePlayerController: AOnlineSamplePlayerController()
{
}

void AOnlineSamplePlayerController::BeginPlay()
{
  Super::BeginPlay();

  ////////////////////////////////////////////////////
  // ONLINE SERVICES
  //
  // Register this player with the Online Services
  UOnlineSampleGameInstance GameInstance = Cast<UOnlineSampleGameInstance>(GetWorld()->G
  UOnlineSampleOnlineSubsystem* OnlineSubsystem = GameInstance->GetOnlineSubsystemOnlineSampleG
  FUniqueNetId* LocalPlayer = Nullptr;//GetLocalPlayer();
  if (LocalPlayer)
  {
    FTaskOfOnlineUserLocalPlayerPlatformUserId * LocalPlayer->GetPlatformUserId();
    if (OnlineSubsystem && !GIsEditor)
    {
      // Read the Title File and display contents on-screen
      FString TitleFileContent = OnlineSubsystem->ReadTitleFileThing("TestuseFile", Loc
      if (GEngine)
      {
        GEngine->AddOnScreenDebugMessage(-1, 10, FColor::Blue, TitleFileContent);
      }
    }
  }

  ////////////////////////////////////////////////////
  // NEXT SECTION...
}

void AOnlineSamplePlayerController::EndPlay(EEndPlayReason::Type EndReason)
{
  Super::EndPlay(EndReason);
}
```

Copy full snippet

Edit Game Mode

You also need to edit the Game Mode source file titled `<PROJECT_NAME>GameMode.cpp` to use the Player Controller Class that you created in the previous section.

OnlineSampleGameMode.cpp

```
// Copyright Epic Games, Inc. All Rights Reserved.

#include "OnlineSampleGameMode.h"
#include "Player/OnlineSamplePlayerController.h"
#include "OnlineSampleCharacter.h"
#include "UObject/ConstructorHelpers.h"

AOnlineSampleGameMode: AOnlineSampleGameMode()
{
  // set default pawn class to our Blueprinted character
  static ConstructorHelpers::FClassFinder<APawn> PlayerPawnBPClass(TEXT("/Game/ThirdPerson
  if (PlayerPawnBPClass != NULL)
  {
    DefaultPawnClass = PlayerPawnBPClass.Class;
  }

  // Assign our new player controller class
  PlayerControllerClass = AOnlineSamplePlayerController::StaticClass();
}
```

Copy full snippet

Build

At this point, you have:

- Enabled and configured the Online Services plugins.
- Appropriately structured your project.
- Added code to implement the Online Services plugins functionality.
- Compiled your project code.

Once you have successfully compiled your project, you are ready to test your project. To test your project, follow these steps:

1. Open your project in the Unreal Editor.
2. Begin Play in Editor to test your project.

> For more information about testing your project, see the Playing and Simulating in Unreal Editor documentation.

Begin Play

This is similar to what you should see when you begin play:

Console Commands

You can also use console commands to debug and test the current online services implementation:

For more information about how to use console commands with the Online Services plugin, see the Online Services Console Commands documentation page.

Command	Description	Example	Output

- Ask questions and help your peers Developer Forums
- Write your own tutorials or read those from others Learning Library